
static keyword

this keyword

Java Object getClass() Method

Java Inheritance

[< prev](#)[next >](#)

Hybrid Inheritance in Java

In Java, **inheritance** is the most important OOPs concept that allows to inherit the properties of a class into another class. In general, it defines Is-A relationship. By using the inheritance feature, we can derive a new class from an existing one. Java supports the following four **types of inheritance**:

- Single Inheritance
- Multi-level Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

In this section, we will discuss only the **hybrid inheritance in Java** with **proper example and different approaches for hybrid inheritance implementation**.

Hybrid Inheritance

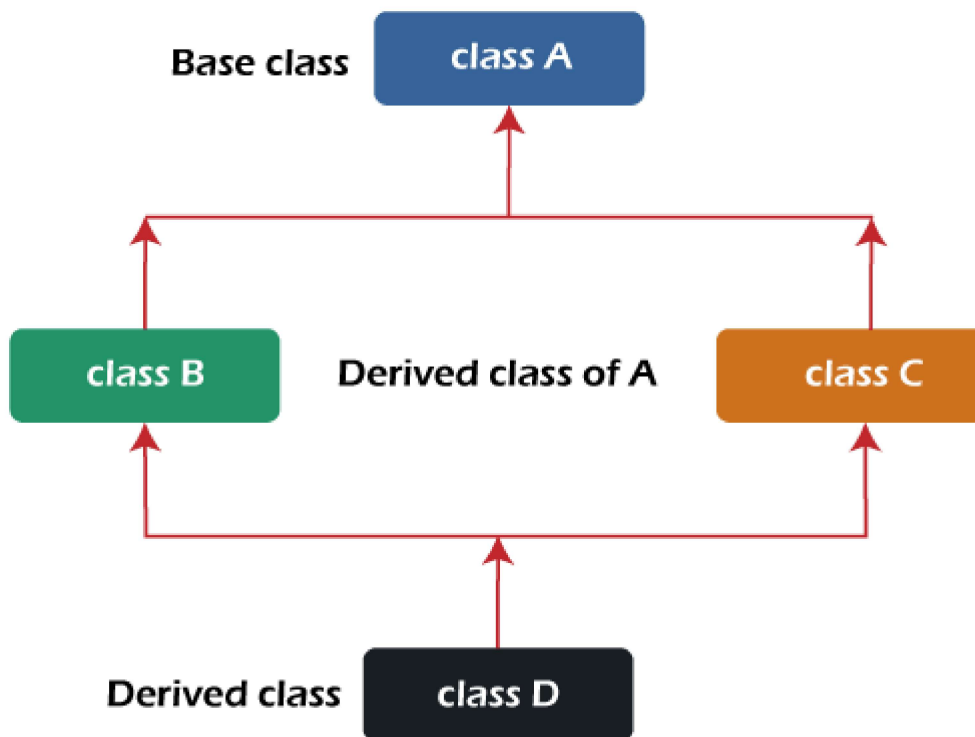
In general, the meaning of hybrid (mixture) is made of more than one thing. In Java, the hybrid inheritance is the composition of two or more types of inheritance. The main purpose of using hybrid inheritance is to modularize the code into well-defined classes. It also provides the code reusability. The hybrid inheritance can be achieved by using the following combinations:

- Single and Multiple Inheritance (not supported but can be achieved through interface)
- Multilevel and Hierarchical Inheritance

- Hierarchical and Single Inheritance
- Multiple and Multilevel Inheritance

How hybrid inheritance works in Java?

For example, there are four classes say A, B, C, and D. Assume that the class "A" and "B" extends the class "C". Also, another class, "D," extends the class "A". Here, the class "A" is a parent class for child class "D" and is also a child class for parent class "C". we can understand the example through pictorial representation.



Single and Multiple Inheritance

In the following Java program, we have achieved the hybrid inheritance by implementing the combination of single and multiple inheritance (through interfaces).

In this program, we have taken the example of Human body that performs different functionalities like eating, walking, talking, dancing etc. Human body may be either Male or Female. So, **Male** and **Female** are the two interfaces of the HumanBody class. Both the child class inherits the functionalities of the HumanBody class that represents the **single** Inheritance.

Suppose, Male and Female may have child. So, the **Child** class also inherits the functionalities of the Male, Female interfaces. It represents the **multiple** inheritance.

The composition of single and multiple inheritance represents the **hybrid** inheritance.

Let's see the implementation.

Child.java

```
class HumanBody
{
    public void displayHuman()
    {
        System.out.println("Method defined inside HumanBody class");
    }
}

interface Male
{
    public void show();
}

interface Female
{
    public void show();
}

public class Child extends HumanBody implements Male, Female
{
    public void show()
    {
        System.out.println("Implementation of show() method defined in interfaces Male and Female");
    }

    public void displayChild()
    {
        System.out.println("Method defined inside Child class");
    }

    public static void main(String args[])
    {
        Child obj = new Child();
        System.out.println("Implementation of Hybrid Inheritance in Java");
        obj.show();
        obj.displayChild();
    }
}
```

```
}  
}
```

Output:

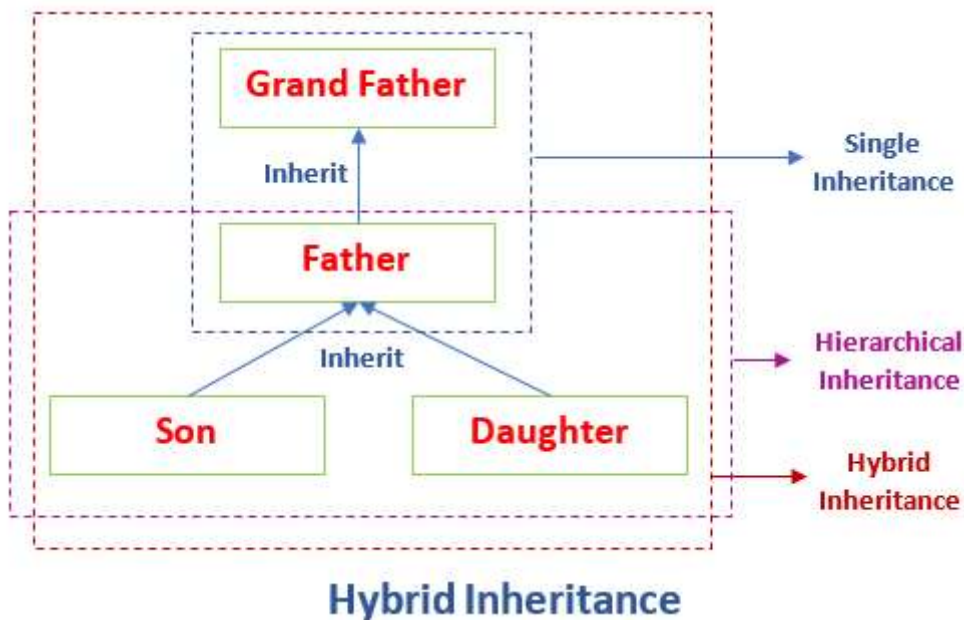
Implementation of Hybrid Inheritance in Java

Implementation of show() method defined in interfaces Male and Female

Method defined inside Child class

Using Multilevel and Hierarchical Inheritance

In the following figure, GrandFather is a super class. The Father class inherits the properties of the GrandFather class. Since Father and GrandFather represents single inheritance. Further, the Father class is inherited by the Son and Daughter class. Thus, the Father becomes the parent class for Son and Daughter. These classes represent the hierarchical inheritance. Combinedly, it denotes the hybrid inheritance.



Let's implement the hybrid inheritance mechanism in a Java program.

Dauhter.java

```
//parent class  
class GrandFather  
{  
    public void showG()  
}
```

```
{
    System.out.println("He is grandfather.");
}
}
//inherits GrandFather properties
class Father extends GrandFather
{
    public void showF()
    {
        System.out.println("He is father.");
    }
}
//inherits Father properties
class Son extends Father
{
    public void showS()
    {
        System.out.println("He is son.");
    }
}
//inherits Father properties
public class Daughter extends Father
{
    public void showD()
    {
        System.out.println("She is daughter.");
    }
    public static void main(String args[])
    {
        //Daughter obj = new Daughter();
        //obj.show();
        Son obj = new Son();
        obj.showS(); // Accessing Son class method
        obj.showF(); // Accessing Father class method
        obj.showG(); // Accessing GrandFather class method
        Daughter obj2 = new Daughter();
        obj2.showD(); // Accessing Daughter class method
    }
}
```

```
obj2.showF(); // Accessing Father class method
obj2.showG(); // Accessing GrandFather class method
}
}
```

Output:

```
He is son.
He is father.
He is grandfather.
She is daughter.
He is father.
He is grandfather.
```

Hierarchical and Single Inheritance

In the following Java program, we have achieved the hybrid inheritance by implementing the combination of hierarchical and single inheritance.

Here class A and class B extends the class C that represents the **hierarchical** inheritance. On the other hand, class D extends the class A that represents the **single** inheritance.

D.java

```
class C
{
    public void disp()
    {
        System.out.println("C");
    }
}

class A extends C
{
    public void disp()
    {
        System.out.println("A");
    }
}
```

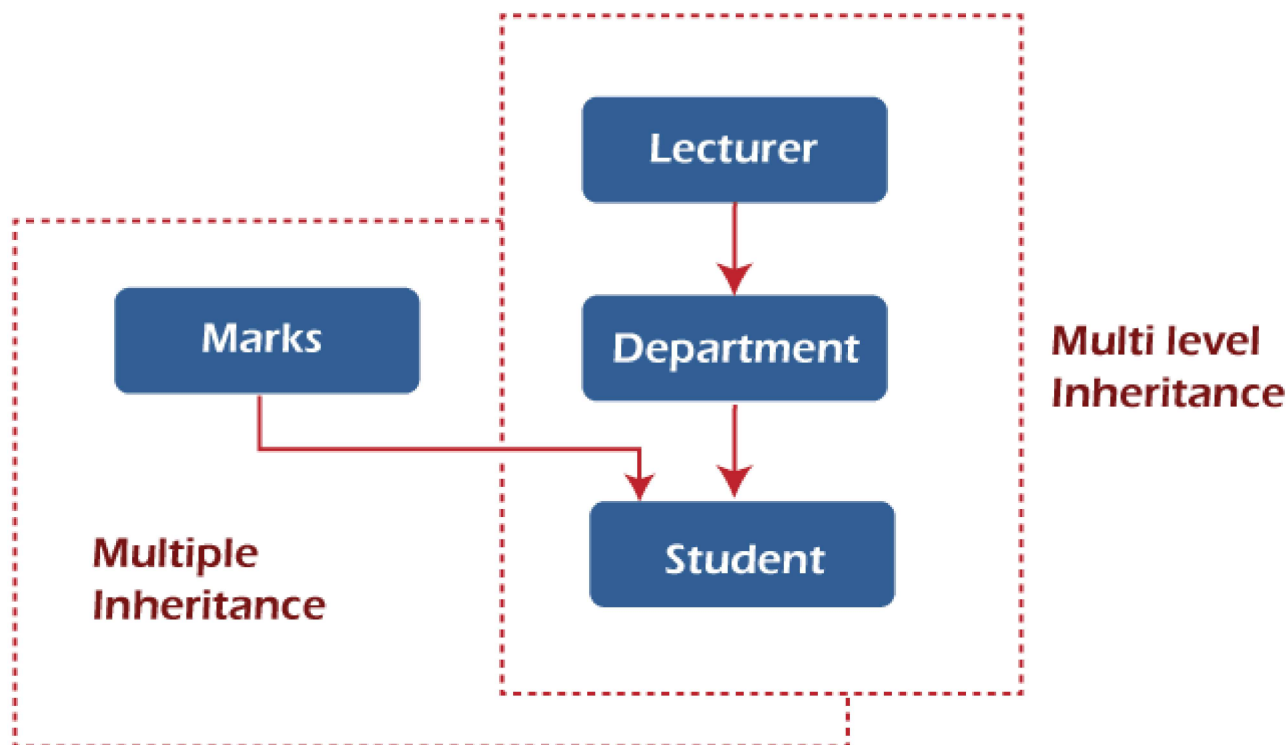
```
}  
class B extends C  
{  
  public void disp()  
  {  
    System.out.println("B");  
  }  
}  
public class D extends A  
{  
  public void disp()  
  {  
    System.out.println("D");  
  }  
  public static void main(String args[])  
  {  
    D obj = new D();  
    obj.disp();  
  }  
}
```

Output:

D

Multiple and Multilevel Inheritance

We can also achieve hybrid inheritance by implementing the multiple and multilevel inheritance. Consider the following figure that depicts an example of hybrid inheritance.



Next Topic [Java IP Address \(IPv4\) Regex Examples](#)

← prev

next →

Latest Courses

